

Experiment – 6

1. **Aim:** To design a Convolution Neural Network (CNN) to classify x-ray images of lungs as COVID infected or not infected.
2. **Objectives:** Students should be familiarize with Learning Architectures and Frameworks using CNN.
3. **Prerequisite:** Python basics
4. **Pre-Experiment Exercise:**
Theory:

CNN:

A convolutional neural network (CNN, or ConvNet) is a class of artificial neural network (ANN), most commonly applied to analyze visual imagery. CNNs are also known as Shift Invariant or Space Invariant Artificial Neural Networks (SIANN), based on the shared-weight architecture of the convolution kernels or filters that slide along input features and provide translation-equivariant responses known as feature maps. Counter-intuitively, most convolutional neural networks are not invariant to translation, due to the down sampling operation they apply to the input. They have applications in image and video recognition, recommender systems, image classification, image segmentation, medical image analysis, natural language processing, brain-computer interfaces, and financial time series.

6. Laboratory Exercise

Procedure

- Take input as any image.
- Process convolution layer on input image to extract features.
- Apply pooling layer to reduce dimension of the image.

Post-Experiments Exercise:

A. Extended Theory:

- a. Explain Architecture of CNN.
- b. Compare RNN , LSTM and CNN

B. Post Lab Program:

- a. Write a program to identify handwritten digits using CNN

C. Conclusion:

1. Write what was performed in the program (s) .
2. What is the significance of program and what Objective is achieved?

D. References:

- [1] <https://stackabuse.com/image-recognition-in-python-with-tensorflow-and-keras/>
- [2] . <https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn>

Laboratory Exercise Procedure:

x-ray image analysis using cnn

Dataset Link:<https://www.kaggle.com/datasets/alifrahman/covid19-chest-xray-image-dataset>

import necessary libraries

```
[2] import numpy as np
import matplotlib.pyplot as plt
import os
import cv2 as cv
import random

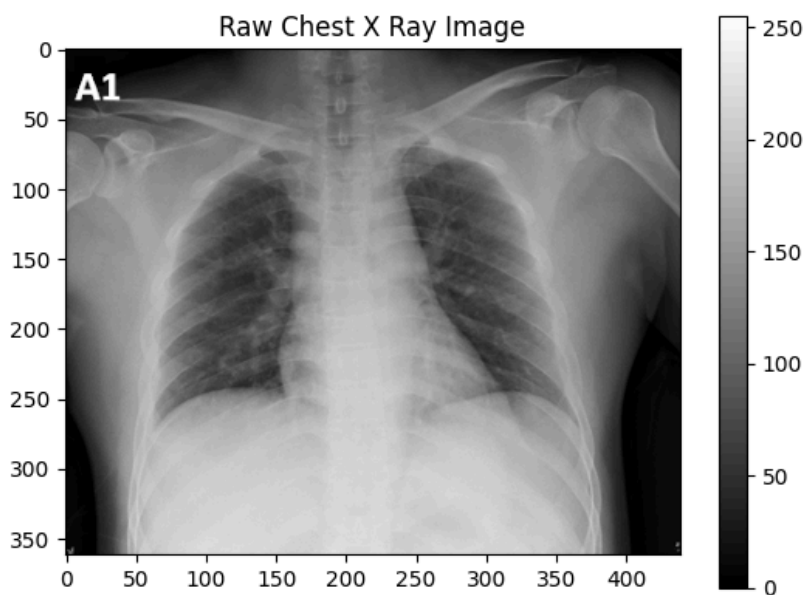
[3] !pip install opendatasets
import opendatasets as od
od.download("https://www.kaggle.com/datasets/alifrahman/covid19-chest-xray-image-dataset")
```

Loading dataset from kaggle

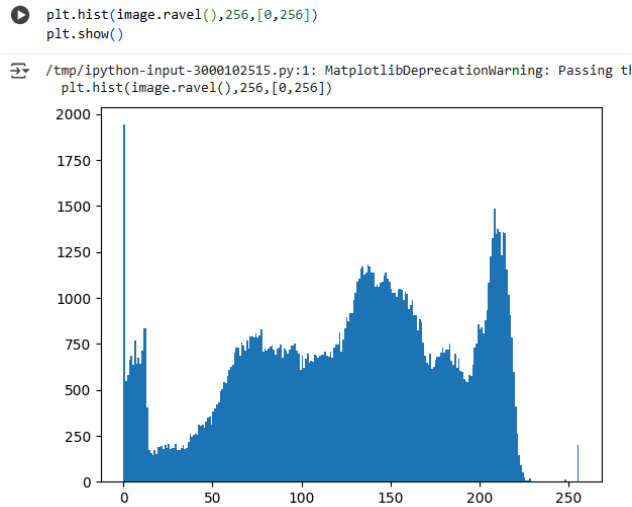
Investigating single image

```
from keras.preprocessing import image
bacteria_path = '/content/covid19-chest-xray-image-dataset/dataset/covid/1-s2.0-S1684118220300682-main.pdf-002-a1.png'
image = cv.imread(bacteria_path, cv.IMREAD_GRAYSCALE)
plt.imshow(image, cmap='gray')
plt.colorbar()
plt.title('Raw Chest X Ray Image')
print(f"The dimensions are {image.shape[0]} pixels height and {image.shape[1]} pixels width")
print(f"The maximum pixel value is {image.max():.4f}")
print(f"The minimum pixel value is {image.min():.4f}")
print(f"The mean value of the pixels is {image.mean():.4f}")
print(f"The standard deviation is {image.std():.4f}")
```

The dimensions are 362 pixels height and 439 pixels width
The maximum pixel value is 255.0000
The minimum pixel value is 0.0000
The mean value of the pixels is 126.5941
The standard deviation is 60.3495



Investigating a single image



Loading images and labels together and resizing images

```
path = '/content/covid19-chest-xray-image-dataset/dataset'
folders=[]
folders = [f for f in sorted(os.listdir(path))]
print(folders)
```

```
['covid', 'normal']
```

```
labels = folders
print (f'The labels are {labels}')
# setting the size of images that we want
image_size = 256
print(f'All images to be resized into {image_size}*{image_size} pixels')
```

The labels are ['covid', 'normal']
All images to be resized into 256*256 pixels

```
# defining a function to load images and labels together
# this function will also resize the images

def load_train(path):
    images = []
    for label in labels:
        direc = os.path.join(path, label)
        class_num = labels.index(label)
        for image_name in os.listdir(direc):
            image_path = os.path.join(direc, image_name)
            image_read = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
            if image_read is not None: # Added check for successful image loading
                image_resized = cv.resize(image_read,(image_size,image_size))
                images.append([image_resized,class_num])
            else:
                print(f'Warning: Could not load image {image_path}') # Add a warning for unreadable images
    return images # Return a list of lists
```

```
train_data = load_train(path)
print(f'Number of loaded images and labels: {len(train_data)}')
```

Number of loaded images and labels: 94

```
#loading the images and labels separately in X and y, to be used later for training
X = []
y = []
for feature, label in train_data: # Iterate through the list of lists
    X.append(feature)
    y.append(label)
```

```
X = np.array(X) # Convert lists to NumPy arrays
y = np.array(y) # Convert lists to NumPy arrays
```

```
print (f'Shape of X = {X.shape}')
print (f'Shape of y = {y.shape}')
```

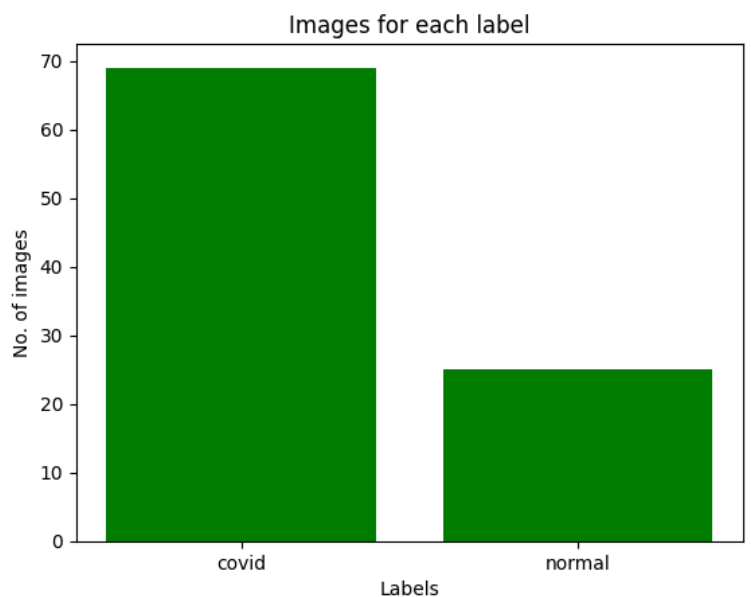
```
Shape of X = (94, 256, 256)
Shape of y = (94,)
```

checking the number of images of each class

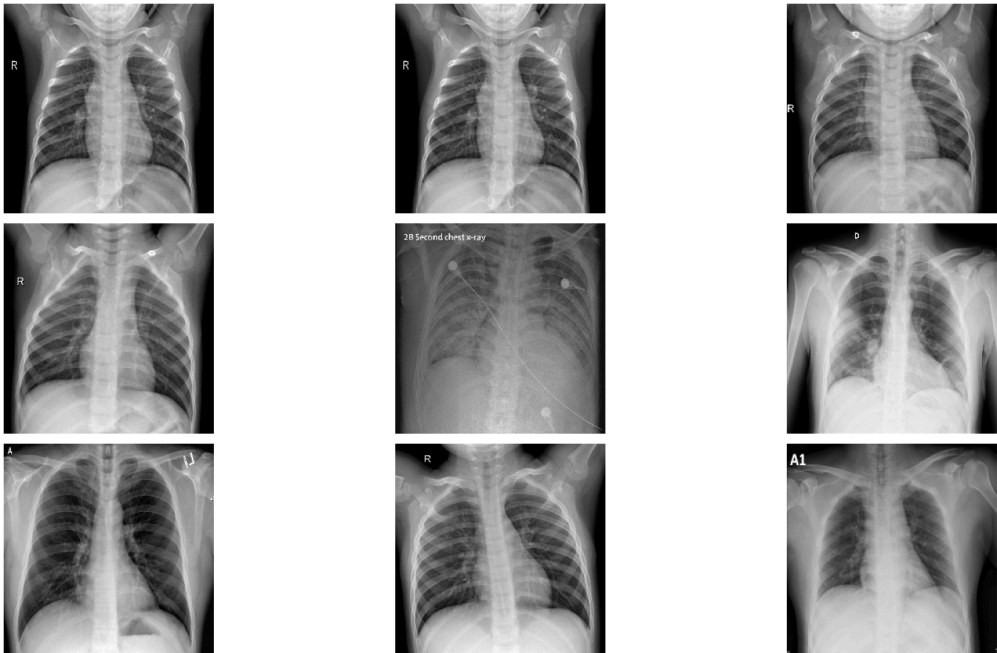
```
a = 0
b = 0
for label in y:
    if label == 0:
        a += 1
    if label == 1:
        b += 1
print (f'Number of Normal images = {a}')
print (f'Number of Covid images = {b}')
```

```
# plotting the data
x_pos = [i for i, _ in enumerate(labels)]
numbers = [a,b]
plt.bar(x_pos,numbers,color = 'green')
plt.xlabel("Labels")
plt.ylabel("No. of images")
plt.title("Images for each label")
plt.xticks(x_pos, labels)
plt.show()
```

```
Number of Normal images = 69
Number of Covid images = 25
```



```
# Displays images
# Extract 9 random images
print('Display Random Images')
# Adjust the size of your images
plt.figure(figsize=(20,10))
for i in range(9):
    num = random.randint(0,len(X)-1)
    plt.subplot(3, 3, i + 1)
    plt.imshow(X[num],cmap='gray')
    plt.axis('off')
# Adjust subplot parameters to give specified padding
plt.tight_layout()
```



Data preprocessing

Normalize the image data by scaling pixel values.

```
# Normalize the image data
X = X / 255.0
print(f'The maximum pixel value after normalization is {X.max():.4f}')
print(f'The minimum pixel value after normalization is {X.min():.4f}')
print(f'Shape of X after normalization = {X.shape}')

The maximum pixel value after normalization is 1.0000
The minimum pixel value after normalization is 0.0000
Shape of X after normalization = (94, 256, 256)
```

Data splitting

Split the data into training and testing sets.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

Shape of X_train: (75, 256, 256)
Shape of X_test: (19, 256, 256)
Shape of y_train: (75,)
Shape of y_test: (19,)
```

Model building

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(image_size, image_size, 1)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 254, 32)	320
max_pooling2d (MaxPooling2D)	(None, 127, 127, 32)	0
conv2d_1 (Conv2D)	(None, 125, 125, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 64)	0
conv2d_2 (Conv2D)	(None, 60, 60, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 30, 30, 128)	0
flatten (Flatten)	(None, 115200)	0
dense (Dense)	(None, 128)	14,745,728
dense_1 (Dense)	(None, 1)	129

Total params: 14,838,529 (56.60 MB)
Trainable params: 14,838,529 (56.60 MB)
Non-trainable params: 0 (0.00 B)

Model Prediction and Comparison

Make predictions on the test data and compare them with the actual labels.

```
# Make predictions on the test set
y_pred_prob = model.predict(X_test)

# Convert predicted probabilities to binary labels (0 or 1) using a threshold of 0.5
y_pred = (y_pred_prob > 0.5).astype(int)

# Print the predicted and actual labels for comparison
print("Predicted labels:\n", y_pred.flatten())
print("\nActual labels:\n", y_test)

1/1 ----- 2s 2s/step
Predicted labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]

Actual labels:
[0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0]

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

Model training

Train the CNN model on the training data.

```
# Reshape the data to include a channel dimension
X_train = X_train.reshape(-1, image_size, image_size, 1)
X_test = X_test.reshape(-1, image_size, image_size, 1)

print(f"Shape of X_train after reshaping: {X_train.shape}")
print(f"Shape of X_test after reshaping: {X_test.shape}")

# Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=256, validation_data=(X_test, y_test))

Shape of X_train after reshaping: (75, 256, 256, 1)
Shape of X_test after reshaping: (19, 256, 256, 1)
Epoch 1/10
1/1 ----- 12s 12s/step - accuracy: 0.2933 - loss: 0.7081 - val_accuracy: 0.8421 - val_loss: 1.5683
Epoch 2/10
1/1 ----- 10s 10s/step - accuracy: 0.7067 - loss: 2.8857 - val_accuracy: 0.8421 - val_loss: 0.4454
Epoch 3/10
1/1 ----- 0s 225ms/step - accuracy: 0.7067 - loss: 0.7358 - val_accuracy: 0.1579 - val_loss: 0.9163
Epoch 4/10
1/1 ----- 0s 218ms/step - accuracy: 0.2933 - loss: 0.8574 - val_accuracy: 1.0000 - val_loss: 0.6371
Epoch 5/10
1/1 ----- 0s 214ms/step - accuracy: 0.9200 - loss: 0.6316 - val_accuracy: 0.8421 - val_loss: 0.4543
Epoch 6/10
1/1 ----- 0s 211ms/step - accuracy: 0.7067 - loss: 0.5416 - val_accuracy: 0.8421 - val_loss: 0.4017
Epoch 7/10
1/1 ----- 0s 213ms/step - accuracy: 0.7067 - loss: 0.5540 - val_accuracy: 0.8421 - val_loss: 0.3951
Epoch 8/10
1/1 ----- 0s 294ms/step - accuracy: 0.7067 - loss: 0.4598 - val_accuracy: 0.9474 - val_loss: 0.3921
Epoch 9/10
1/1 ----- 0s 213ms/step - accuracy: 0.9600 - loss: 0.4141 - val_accuracy: 0.8947 - val_loss: 0.2852
Epoch 10/10
1/1 ----- 0s 213ms/step - accuracy: 0.9467 - loss: 0.3236 - val_accuracy: 0.8947 - val_loss: 0.2173
```

Model evaluation

Evaluate the performance of the trained model on the testing data.

```
# Define a function to load and preprocess a single image
def preprocess_single_image(image_path, image_size):
    img = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
    if img is not None:
        img_resized = cv.resize(img, (image_size, image_size))
        img_normalized = img_resized / 255.0 # Normalize pixel values
        img_resized = img_normalized.reshape(1, image_size, image_size, 1) # Reshape for model input
        return img_resized
    else:
        return None

# Specify the path to the image you want to test
single_image_path = '/content/covid19-chest-xray-image-dataset/dataset/covid/1-s2.0-S0929664620300449-gr2_lrg-c.jpg'

# Preprocess the image
preprocessed_image = preprocess_single_image(single_image_path, image_size)

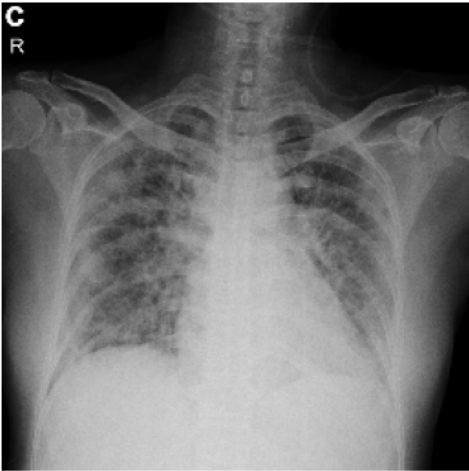
if preprocessed_image is not None:
    # Make a prediction
    prediction_prob = model.predict(preprocessed_image)
    prediction_class = (prediction_prob > 0.5).astype(int)

    # Map the prediction to the actual label
    predicted_label = labels[prediction_class[0][0]]

    # Display the image and the prediction
    plt.imshow(preprocessed_image.reshape(image_size, image_size), cmap='gray')
    plt.title(f'Predicted: {predicted_label}')
    plt.axis('off')
    plt.show()

    print(f"The model predicts this image is: {predicted_label}")
else:
    print(f"Could not preprocess image from {single_image_path}")

1/1 ----- 1s 807ms/step
Predicted: covid
```



The model predicts this image is: covid

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)

print(f'\nTest loss: {test_loss:.4f}')
print(f'Test accuracy: {test_acc:.4f}')

1/1 - 0s - 66ms/step - accuracy: 0.8947 - loss: 0.2173

Test loss: 0.2173
Test accuracy: 0.8947

Post Experiment Exercise:

Dataset: MNIST Handwritten Digits Dataset

Step 1: Import required libraries

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D,

MaxPooling2D, Flatten, Dense

from tensorflow.keras.utils import to_categorical

Step 2: Load the MNIST dataset

(x_train, y_train), (x_test, y_test) = mnist.load_

Step 3: Preprocess the data

Reshape the data to include channel dimension

(grayscale -> 1 channel)

x_train = x_train.reshape(x_train.shape[0], 28,

x_test = x_test.reshape(x_test.shape[0], 28, 28,

Normalize pixel values (0-255) to (0-1)

x_train = x_train.astype('float32') / 255

x_test = x_test.astype('float32') / 255

Convert labels to one-hot encoded format

y_train = to_categorical(y_train)

y_test = to_categorical(y_test)

Step 4: Build the CNN model

model = Sequential()

Convolutional layer 1

model.add(Conv2D(32, kernel_size=(3,3),

activation='relu', input_shape=(28,28,1)))

Pooling layer 1

model.add(MaxPooling2D(pool_size=(2,2)))

Convolutional layer 2

model.add(Conv2D(64, kernel_size=(3,3),

activation='relu'))

Pooling layer 2

model.add(MaxPooling2D(pool_size=(2,2)))

Flatten the feature maps

model.add(Flatten())

Fully connected layer

model.add(Dense(128, activation='relu'))

Output layer with 10 neurons (digits 0-9)

model.add(Dense(10, activation='softmax'))

Step 5: Compile the model

model.compile(optimizer='adam',

loss='categorical_crossentropy', metrics=['accuracy'])

Step 6: Train the model

print("Training the CNN model...")

model.fit(x_train, y_train, epochs=5, batch_size=128,

validation_data=(x_test, y_test))

Step 7: Evaluate the model

test_loss, test_acc = model.evaluate(x_test, y_test,

verbose=0)

print("\nTest Accuracy:", test_acc)

Step 8: Visualize predictions

predictions = model.predict(x_test[:9])

plt.figure(figsize=(10, 5))

for i in range(9):

plt.subplot(3, 3, i+1)

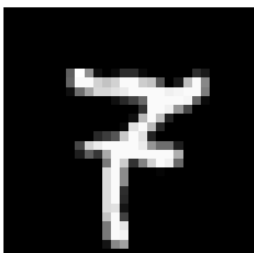
plt.imshow(x_test[i].reshape(28,28), cmap='gray')

plt.title(f"Predicted: {np.argmax(predictions[i])}")

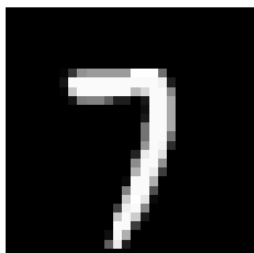
plt.axis('off')

plt.show()

Actual: 7
Predicted: 7



Actual: 7
Predicted: 7



Actual: 3
Predicted: 3



Actual: 3
Predicted: 3



Actual: 4
Predicted: 4

